

Project Report: Designing and Implementing a Simplified Data Warehouse Using open library book data

Dataset Overview

The Open Library dataset contains several files that offer detailed metadata related to books, their ratings, authors, and editions. This report examines the following specific files: **works**, **ratings**, **editions**, and **authors**.

Dataset Structure

- **File Format:** JSON
 - **Primary Files:**
 - **Works:** This file holds information about books, such as titles, authors, subjects, and additional metadata.
 - **Ratings:** This file provides ratings for books, with a link to the corresponding work via the `work_id`.
 - **Editions:** This file includes data on various editions of works, with details about each edition.
 - **Authors:** This file contains data on authors, including their names and the works they are linked to.
-

Key Fields

1. **Works Dataset:**
 - `work_id`: Unique identifier for each work.
 - `title`: Title of the work.
 - `author_id`: List of author IDs associated with the work (foreign key referencing `dim_authors`).
 - `revision`: Topics or genres of the work updates.
 - `covers`: Topics or genres of the work.
2. **Ratings Dataset:**
 - `work_id`: Links the rating to a specific work (foreign key referencing `dim_works`).
 - `rating`: Numerical rating assigned to the work.
 - `rating_count`: The number of ratings the work has received.
 - `date`: Date when the rating was given.
3. **Editions Dataset:**
 - `book_id`: Unique identifier for each edition (primary key).

- **work_id:** Links the edition to a work (foreign key referencing `dim_works`).
 - **title:** Title of the edition.
 - **authors:** Authors associated with this specific edition.
 - **publish_date:** Publication date of the edition.
 - **isbn_13:** ISBN-13 of the book edition.
 - **physical_format:** Format of the book edition (e.g., hardcover, paperback).
4. **Authors Dataset:**
- **author_id:** Unique identifier for each author (primary key).
 - **name:** Name of the author.
 - **revision:** The revision number of the author's data.
 - **created_at:** Timestamp when the author data was first created.
 - **key:** A unique key for the author (likely from an external system like Open Library).
 - **source_records:** Source records associated with the author (can be a list of source identifiers).
 - **latest_revision:** Latest revision number for the author's data.
 - **last_modified:** Timestamp of when the author's data was last modified.
-

Dataset Content

- **Works Dataset:** Contains comprehensive metadata about books in the Open Library catalog, with key fields for book titles, subjects, and author information.
 - **Ratings Dataset:** Contains ratings for works; however, many works may lack ratings due to insufficient data or a lack of ratings altogether.
 - **Editions Dataset:** Includes various editions of books, providing more detailed data about specific versions of a work.
 - **Authors Dataset:** Contains author information, linking them to the works they have created or contributed to.
-

Initial Observations

1. **Missing Data:**
 - The **works dataset** may have missing fields for subjects or authors, and some works may not have a `published_date`.
 - The **ratings dataset** may lack rating values for certain works, potentially due to insufficient data or the absence of ratings.
 - The **editions dataset** may have missing or inconsistent data, particularly with respect to authors or publication dates.
 - The **authors dataset** may be incomplete, as some authors may not have associated works listed.
2. **Data Types:**

- There are inconsistent formats, especially with date fields such as `published_date`, which are stored as strings and will need to be converted into proper date types.
 - In the **ratings dataset**, both `rating` and `rating_count` are numeric but may be stored as strings, requiring conversion.
 - 3. **Nested Structures:**
 - The `authors` field in the **works** and **editions** datasets is a list of IDs, which will need to be linked to the **authors** dataset for proper association.
 - The `subjects` field in the **works** dataset is a list of strings, requiring transformation into a more normalized structure for efficient querying.
 - 4. **Duplicates:**
 - Preliminary checks suggest no obvious duplicate rows in the datasets. However, duplicates across tables (e.g., multiple editions of the same work) may require handling during the ETL (Extract, Transform, Load) process.
 - 5. **Data Quality:**
 - Certain fields are either missing or have inconsistent formats (e.g., `published_date` and `rating`). These fields will need to be cleaned and standardized during the ETL process.
-

Dimensional Modeling

To design a simplified star schema for the Open Library dataset, the focus will be on the fact table and the associated dimension tables for **authors**, **works**, **ratings**, and **editions**.

1. Star Schema Design

The star schema will consist of a central fact table (`fact_book_rating`) surrounded by dimension tables. The structure of the schema includes:

Fact Table

- **fact_book_ratings:** Contains aggregated data about books, with references to associated works, editions, ratings, and authors.
 - `book_id` (Foreign Key to `dim_editions`): Refers to the book edition associated with the rating.
 - `title` :name of the title of the book
 - `work_id` (Foreign Key to `dim_works`): Refers to the work associated with the book.
 - `book_info_id` (Primary Key): Unique identifier for each book record.
 - `author_id` (Foreign Key to `dim_authors`): Refers to the author(s) associated with the book.

- `author_name`: Name of the author associated with the book.
- `rating`: Numeric score representing the rating given to the book.

Dimension Tables

- **dim_editions**: Stores information about book editions.
 - `book_id` (Primary Key): Unique identifier for each book edition.
 - `title`: Title of the book edition.
 - `isbn_13`: ISBN-13 of the book edition.
 - `isbn_10`: ISBN-10 of the book edition.
 - `publishers`: Publisher of the book edition.
 - `publish_date`: Date when the book was published.
 - `physical_format`: Format of the book edition (e.g., hardcover, paperback).
 - `number_of_pages`: Total number of pages in the book edition.
 - `authors`: Authors of the book edition.
 - `type`: Type of edition (e.g., first edition, revised edition).
 - `revision`: Revision number of the book edition.
 - `last_modified`: Timestamp of the last modification.
- **dim_works**: Stores information about literary works.
 - `work_id` (Primary Key): Unique identifier for each work.
 - `title`: Title of the work.
 - `author_id` (Foreign Key to `dim_authors`): Identifier(s) of the author(s) of the work.
 - `subject`: Subject(s) of the work (may include multiple subjects).
 - `created_at`: Date when the work was created.
- **dim_ratings**: Stores ratings information for books.
 - `rating_id` (Primary Key): Unique identifier for each rating.
 - `book_id` (Foreign Key to `dim_editions`): Identifier for the book being rated.
 - `work_id` (Foreign Key to `dim_works`): Identifier for the work associated with the rating.
 - `rating`: Rating score given to the book.
 - `date`: Date when the rating was given.
- **dim_authors**: Stores information about authors.
 - `author_id` (Primary Key): Unique identifier for each author.
 - `name`: Name of the author.
 - `revision`: The revision number of the author's data.
 - `created_at`: Timestamp when the author data was first created.
 - `key`: A unique key for the author (likely from an external system like Open Library).
 - `source_records`: Source records associated with the author (can be a list of source identifiers).
 - `latest_revision`: Latest revision number for the author's data.

- o last_modified: Timestamp of when the author's data was last modified.

2) Table Schema Definitions

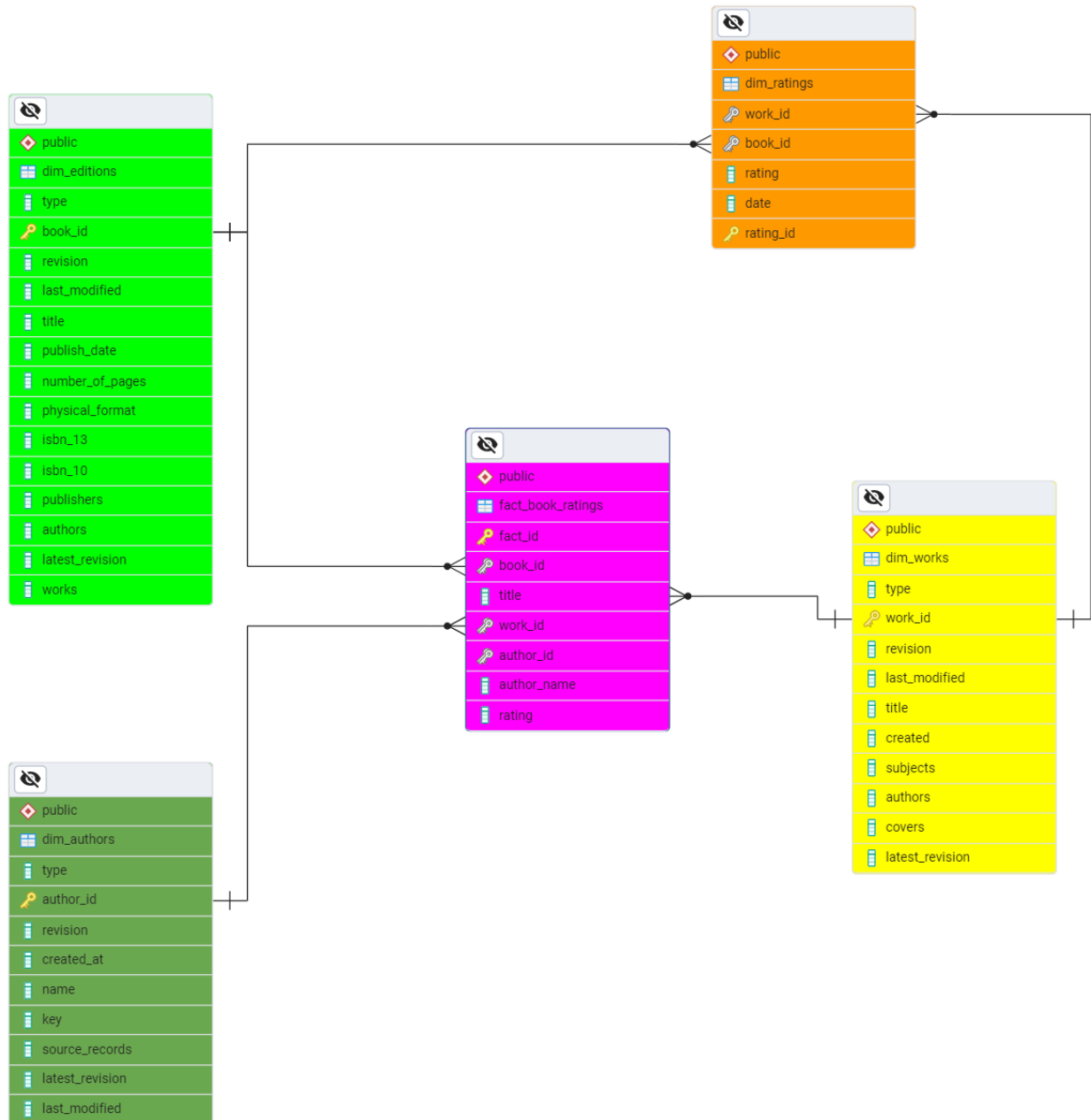
```
CREATE TABLE fact_book_ratings
(
    book_id VARCHAR,
    title VARCHAR(255),
    work_id VARCHAR,
    author_id VARCHAR,
    author_name VARCHAR(255),
    rating NUMERIC,
    CONSTRAINT fact_book_ratings_author_id_fkey FOREIGN KEY (author_id)
        REFERENCES dim_authors (author_id) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fact_book_ratings_book_id_fkey FOREIGN KEY (book_id)
        REFERENCES dim_editions (book_id) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fact_book_ratings_work_id_fkey FOREIGN KEY (work_id)
        REFERENCES dim_works (work_id) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
CREATE TABLE dim_editions
(
    type VARCHAR(255),,
    book_id VARCHAR(255),NOT NULL,
    revision BIGINT,
    last_modified TIMESTAMP WITHOUT TIME ZONE,
    title VARCHAR(255),,
    publish_date TIMESTAMP WITHOUT TIME ZONE,
    number_of_pages DOUBLE PRECISION,
    physical_format VARCHAR,
    isbn_13 VARCHAR(255),,
    isbn_10 VARCHAR(255),,
    publishers VARCHAR,
    authors VARCHAR(255),,
    latest_revision DOUBLE PRECISION,
    works VARCHAR(255),,
    CONSTRAINT dim_editions_pkey PRIMARY KEY (book_id)
);
CREATE TABLE dim_ratings
(
    work_id VARCHAR,
    book_id VARCHAR,
    rating BIGINT,
    date TIMESTAMP WITHOUT TIME ZONE,
    rating_id BIGINT NOT NULL DEFAULT
nextval('dim_ratings_rating_id_seq'::regclass),
    CONSTRAINT dim_ratings_pkey PRIMARY KEY (rating_id),
    CONSTRAINT dim_ratings_work_id_fkey FOREIGN KEY (work_id)
```

```

        REFERENCES dim_works (work_id) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE
        NOT VALID,
    CONSTRAINT g FOREIGN KEY (book_id)
        REFERENCES dim_editions (book_id) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE
        NOT VALID
);
CREATE TABLE dim_works
(
    type VARCHAR,
    work_id VARCHAR NOT NULL,
    revision BIGINT,
    last_modified TIMESTAMP WITHOUT TIME ZONE,
    title VARCHAR,
    created TIMESTAMP WITHOUT TIME ZONE,
    subjects VARCHAR,
    authors VARCHAR,
    covers VARCHAR,
    latest_revision BIGINT,
    CONSTRAINT dim_works_pkey PRIMARY KEY (work_id)
);
CREATE TABLE dim_authors
(
    type VARCHAR,
    author_id VARCHAR NOT NULL,
    revision BIGINT,
    created_at TIMESTAMP WITHOUT TIME ZONE,
    name VARCHAR,
    key VARCHAR,
    source_records VARCHAR,
    latest_revision DOUBLE PRECISION,
    last_modified TIMESTAMP WITHOUT TIME ZONE,
    CONSTRAINT dim_authors_pkey PRIMARY KEY (author_id)
);

```

3 STAR schema



3. ETL Design:

```
1 import pandas as pd
2 from sqlalchemy import create_engine
3 from sqlalchemy.exc import SQLAlchemyError
4 import os
5 import json
6 import re
7
8 # Specify the path to your file
9 FILE_PATH = r"D:\datasets\ol_dump_editions_2025-03-31.txt"
10
11 # PostgreSQL database connection parameters
12 DB_PARAMS = {
13     'dbname': 'Second DB',
14     'user': 'postgres',
15     'password': '1256', # Ensure this password is correct
16     'host': 'localhost',
17     'port': '5432'
18 }
19
20 def extract_data(file_path):
21     """Extract data from the editions dump file."""
22     data = []
23     try:
24         with open(file_path, 'r', encoding='utf-8') as file:
```

2

```
19
20
21 def extract_data(file_path):
22     """Extract data from the editions dump file."""
23     data = []
24     try:
25         with open(file_path, 'r', encoding='utf-8') as file:
26             for line in file:
27                 parts = line.strip().split('\t')
28                 if len(parts) >= 5: # Ensure there are enough columns
29                     json_data = json.loads(parts[4]) # Parse the JSON part
30                     data.append({
31                         'type': parts[0],
32                         'book_id': parts[1],
33                         'revision': int(parts[2]),
34                         'last_modified': parts[3],
35                         'title': json_data.get('title'),
36                         'publish_date': json_data.get('publish_date'),
37                         'number_of_pages': json_data.get('number_of_pages'),
38                         'physical_format': json_data.get('physical_format'),
39                         'isbn_13': json_data.get('isbn_13'),
40                         'isbn_10': json_data.get('isbn_10'),
41                         'publishers': json_data.get('publishers'),
42                         'authors': json_data.get('authors'),
43                         'latest_revision': json_data.get('latest_revision').
```

3

```
59 def clean_data(data):
60     """Clean the extracted data."""
61     try:
62         # Clean the data by removing extra characters
63         data['type'] = data['type'].replace('/type/', '', regex=True)
64         data['book_id'] = data['book_id'].replace('/books/', '', regex=True)
65
66         # Clean JSON-like fields to remove extra quotes and commas
67         data['isbn_13'] = data['isbn_13'].apply(
68             lambda x: ', '.join([isbn.strip('"') for isbn in x]) if isinstance(x, list)
69         )
70         data['isbn_10'] = data['isbn_10'].apply(
71             lambda x: ', '.join([isbn.strip('"') for isbn in x]) if isinstance(x, list)
72         )
73         data['publishers'] = data['publishers'].apply(
74             lambda x: ', '.join([publisher.strip('"') for publisher in x]) if isinstance(x, list)
75         )
76         data['authors'] = data['authors'].apply(
77             lambda x: ', '.join([author.get('key').replace('/authors/', '') for author in x]) if isinstance(x, list)
78         )
79
80         data['works'] = data['works'].apply(
81             lambda x: ', '.join([work.get('key').replace('/works/', '') for work in x]) if isinstance(x, list)
82         )
83
84         # Clean date formats
85         data['last_modified'] = pd.to_datetime(data['last_modified'], errors='coerce')
86         data['publish_date'] = pd.to_datetime(data['publish_date'], errors='coerce')
87
88         # Remove duplicates based on 'book_id'
89         data = data.drop_duplicates(subset=['book_id'])
```

4

```
115 def transform_data(data):
116     return pd.DataFrame()
117
118 def load_data(data, db_params):
119     """Load data into PostgreSQL database."""
120     try:
121         engine = create_engine(f'postgresql://{db_params["user"]}({db_params["password"]})@{db_params["host"]}({db_params["port"]}({db_params["dbname"]})')
122
123         data.to_sql('ol_dump_editions', con=engine, if_exists='replace', index=False, chunksize=500)
124         print("Data loaded successfully into the database!")
125     except SQLAlchemyError as e:
126         print(f"Database error: {e}")
127     except Exception as e:
128         print(f"An error occurred while loading data: {e}")
129
130 def main():
131     """Main function to run the ETL process."""
132     if not os.path.exists(FILE_PATH):
133         print(f"Error: The specified file (FILE_PATH) was not found.")
134
135     # Extract data
136     data = extract_data(FILE_PATH)
137
138     # Clean data
139     data = clean_data(data)
140
141     # Transform and load data
142     transformed_data = transform_data(data)
143     load_data(transformed_data, DB_PARAMS)
```



```
def extract_data(file_path):
    ...
    'last_modified': json_data.get('last_modified', {}).get('value')
    ...
    if len(data) >= 500000: # limit to 500,000 rows
        break

Run authors EETL.py

C:\Users\Haisha\PycharmProjects\pythonProject\venv\Scripts\python.exe "C:\Users\Haisha\PycharmProjects\Data Mining\authors EETL.py"
type ... last_modified
0 /type/author ... 2021-12-26T21:42:09.238563
1 /type/author ... 2021-12-26T22:24:12.304187
2 /type/author ... 2021-12-26T23:32:38.436866
3 /type/author ... 2021-12-27T00:25:13.211432
4 /type/author ... 2021-12-27T00:25:45.312956
...
499995 /type/author ... 2022-11-22T00:24:07.209988
499996 /type/author ... 2022-11-22T00:25:32.944356
499997 /type/author ... 2022-11-22T00:31:34.553625
499998 /type/author ... 2022-11-22T00:40:50.280372
499999 /type/author ... 2022-11-22T00:53:46.568372

[500000 rows x 9 columns]
Extracted rows: 500000
Purified Total rows: 499911
Data loaded successfully into the database!
Dropped duplicate rows: 399917

Process finished with exit code 0
```

n

```
def extract_data(file_path):
    ...
    'works': json_data.get('works')
    ...
    if len(data) >= 500000: # limit to 400,000 rows (adjust as needed)
        break

Run edition EETL.py

C:\Users\Haisha\PycharmProjects\pythonProject\venv\Scripts\python.exe "C:\Users\Haisha\PycharmProjects\Data Mining\edition EETL.py"
type ... works
0 /type/edition ... [{"key": "/works/OL8951147W"}]
1 /type/edition ... [{"key": "/works/OL14982799W"}]
2 /type/edition ... [{"key": "/works/OL14982799W"}]
3 /type/edition ... [{"key": "/works/OL1498191W"}]
4 /type/edition ... [{"key": "/works/OL28282135W"}]
...
499995 /type/edition ... [{"key": "/works/OL22412791W"}]
499996 /type/edition ... [{"key": "/works/OL9451007W"}]
499997 /type/edition ... [{"key": "/works/OL22413213W"}]
499998 /type/edition ... [{"key": "/works/OL22413464W"}]
499999 /type/edition ... [{"key": "/works/OL22413493W"}]

[500000 rows x 14 columns]
Extracted Rows: 500000
Data cleaned successfully!
No new data to load (all records were duplicates).

Process finished with exit code 0
```

```
def clean_data(data):
    ...
    # Drop missing or empty work_id or book_id
    data = data.dropna(subset=['work_id', 'book_id'])
    data = data[data['book_id'] != ''] # Discard rows with null or empty book_id

Run rating EETL.py

C:\Users\Haisha\PycharmProjects\pythonProject\venv\Scripts\python.exe "C:\Users\Haisha\PycharmProjects\Data Mining\rating EETL.py"
work_id book_id rating date
1 OL1629179W OL22981670H 5 2018-06-20
2 OL4220833W OL10690412H 5 2018-06-20
3 OL5264255W OL2719185H 5 2018-06-20
4 OL1681415W OL2582724H 5 2018-06-20
6 OL1853596W OL2331075H 2 2018-06-20
...
399995 OL28765867W OL10690590H 3 2023-01-18
399996 OL20802694W OL28419727H 5 2023-01-18
399997 OL4320394H OL2838669H 5 2023-01-18
399998 OL5895273W OL3472304H 4 2023-01-18
399999 OL5267298H OL25435142H 1 2023-01-18

[342856 rows x 4 columns]
Purified Data Rows: 342856
Data loaded successfully into the database!

Process finished with exit code 0
```

The screenshot shows a VS Code editor with a project named 'Data Mining'. The file explorer on the left shows a directory structure with files like 'authors EETL.py', 'cleaning_errors.log', 'edition EETL.py', 'rating EETL.py', 'works EETL.py', 'Y-B visualization.py', and 'Rating visualization.py'. The main editor window displays the 'extract_data' function in 'edition EETL.py'. The function takes a file path and extracts data from a JSON file, limiting it to 520,000 rows. The output console shows the execution results for 'authors EETL.py'.

```
def extract_data(file_path):
    with open(file_path, 'r', encoding='utf-8') as file:
        'source_records': json_data.get('source_records', []), # Ensure empty list if None
        'latest_revision': json_data.get('latest_revision'),
        'last_modified': json_data.get('last_modified', {}).get('value')
    })
    if len(data) >= 520000: # Limit to 500,000 rows
        break
    print(pd.DataFrame(data))
    print(f"Extracted rows : {len(data)} ")
    return pd.DataFrame(data)
except FileNotFoundError:
    print(f"Error: The specified file (file_path) was not found.")
```

Run Output (authors EETL.py):

```
519995 /type/author ... 2008-04-01T03:28:50.625462
519996 /type/author ... 2008-09-06T23:21:26.076874
519997 /type/author ... 2008-08-26T23:28:33.098956
519998 /type/author ... 2008-08-26T04:40:53.780154
519999 /type/author ... 2008-04-01T03:28:50.625462

[520000 rows x 9 columns]
Extracted rows : 520000
Purified Total rows: 519901
No new data to load (all records were duplicates).
Dropped duplicate rows: 519901

Process finished with exit code 0
```

The screenshot shows the same VS Code editor with the 'works EETL.py' file selected. The main editor window displays the 'extract_works_data' function, which extracts data from a JSON file and limits it to 510,000 rows. The output console shows the execution results for 'works EETL.py'.

```
def extract_works_data(file_path):
    with open(file_path, 'r', encoding='utf-8') as file:
        covers = json.dumps(raw_json.get('covers', []))
        'latest_revision': raw_json.get('latest_revision')
    })
    except json.JSONDecodeError:
        continue # Skip invalid JSON lines
    if len(records) >= 510000:
        break
    print(pd.DataFrame(records))
    print(f"Extracted rows : {len(records)} ")
    return pd.DataFrame(records)
```

Run Output (works EETL.py):

```
509995 work OL24162802W ... [10573263] 2
509996 work OL24162972W ... [10573668] 2
509997 work OL24162991W ... [10573719] 2
509998 work OL241630W ... [2506573] 3
509999 work OL24163161W ... [10574157] 1

[510000 rows x 10 columns]
Extracted rows : 510000
Purified Data Rows : 480603
Data loaded successfully into the database!
Dropped duplicate rows: 376715

Process finished with exit code 0
```

2: Data Loading and Analysis

1. Data Warehouse Implementation and Loading:

The screenshot displays two instances of the pgAdmin 4 interface, showing SQL queries and their results in a data warehouse.

Top Query:

```
1 select * from dim_works ;
2 select * from dim_authors;
3 select *from dim_ratings;
4 select * from dim_editions;
5 select *from fact_book_ratings;
6
7
8 --relating authors works
9 SELECT *
10 FROM dim_authors
11 JOIN dim_works
12 ON dim_authors.author_id = dim_works.authors;
13
```

Top Result Set:

	type	author_id	revision	created_at	name	key	source_records
	text	[PK] text	bigint	timestamp without time zone	text	text	text
1	/type/author	OL10000226A	1	2021-12-26 21:42:09.238563	Alan R. Blackmer	OL10000226A	(bwb:9780674593473)
2	/type/author	OL10000440A	1	2021-12-26 22:24:12.304187	Carmen Melillo Sanz	OL10000440A	(bwb:9788490150221)
3	/type/author	OL10000917A	1	2021-12-26 23:32:38.430686	Tom Popkewitz	OL10000917A	(bwb:9781607527602)
4	/type/author	OL10001265A	1	2021-12-27 00:25:38.211432	Fan Chengda	OL10001265A	(bwb:9780295802060)
5	/type/author	OL10001270A	1	2021-12-27 00:25:45.312956	Mario R. Dinunzio	OL10001270A	(bwb:9781604264043)
6	/type/author	OL10001396A	1	2021-12-27 00:57:39.652969	Thomas S. O'Connor	OL10001396A	(bwb:9781606493557)
7	/type/author	OL10001709A	1	2021-12-27 01:38:04.408499	Jon Vriesacker	OL10001709A	(bwb:9781480371354)
8	/type/author	OL10001836A	1	2021-12-27 01:48:07.23888	Peter Liewuen	OL10001836A	(bwb:9781476871578)
9	/type/author	OL10002328A	1	2021-12-27 02:48:53.242959	Yvonne Velasquez	OL10002328A	(bwb:9780578158846)
10	/type/author	OL10002401A	1	2021-12-27 03:10:47.846705	J. Bianchi	OL10002401A	(bwb:978075047706761)

Total rows: 499911 Query complete 00:00:01.798

Bottom Query:

```
1 select * from dim_works ;
2 select * from dim_authors;
3 select *from dim_ratings;
4 select * from dim_editions;
5 select *from fact_book_ratings;
6
7
8 --relating authors works
9 SELECT *
10 FROM dim_authors
11 JOIN dim_works
12 ON dim_authors.author_id = dim_works.authors;
13
```

Bottom Result Set:

	type	book_id	revision	last_modified	title
	text	[PK] text	bigint	timestamp without time zone	text
1	edition	OL10000241M	3	2010-04-13 02:52:51.925831	The National Register of Apparel Manufacturers, Men & Boys
2	edition	OL10000488M	3	2011-04-28 06:17:32.348931	Public Processions Etc. (Northern Ireland) Bill [H.L.]
3	edition	OL10000723M	4	2022-07-17 04:38:18.718745	Crime and Disorder Bill [H.L.]
4	edition	OL10001042M	3	2011-04-28 19:27:40.205073	Northern Ireland Bill
5	edition	OL1000267M	11	2024-08-06 14:00:42.514369	The psalm killer
6	edition	OL10002693M	3	2011-06-08 04:28:03.025393	Health Authorities Bill
7	edition	OL10003167M	3	2011-04-28 00:52:09.260425	Nursery Education and Grant-Maintained Schools Bill
8	edition	OL10003490M	7	2022-07-17 05:19:12.575779	Arbitration Bill [Lords] (Parliamentary Debates: [1995-96])
9	edition	OL10003761M	4	2022-07-17 07:01:12.718767	Finance Bill (Except Clauses 2, 28 and 99) (Parliamentary Debates: [1998-99])
10	edition	OL10004076M	4	2022-07-17 07:01:02.910777	Welfare Reform and Domestic Bill (Parliamentary Debates: [1998-99])

Total rows: 431083 Query complete 00:00:02.693

File Object Tools Edit View Window Help

Welcome perfect last.sql* x hhhhhh.sql* x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr...

Second DB/postgres@Projects

Query Query History

```
select *from fact_book_ratings;
```

Data Output Messages Notifications

Showing rows: 1 to 259 Page No: 1 of 1

	book_id character varying	title character varying (255)	work_id character varying	author_id character varying	author_name character varying (255)	rating numeric
1	OL11353750M	Sacramentary	OL7893135W	OL2589641A	Catholic	3
2	OL11466390M	What the Bible is All about	OL17048540W	OL20625A	Henrietta C. Mears	1
3	OL15233460M	De superman au surhomme	OL15653145W	OL20735A	Umberto Eco	3
4	OL15233460M	De superman au surhomme	OL15653145W	OL20735A	Umberto Eco	4
5	OL17286871M	Yanlis okumalar	OL15653145W	OL20735A	Umberto Eco	3
6	OL17286871M	Yanlis okumalar	OL15653145W	OL20735A	Umberto Eco	4
7	OL25065956M	Strongest and weakest	OL9592680W	OL1394228A	Camilla De la Bédoyère	4
8	OL25299695M	Democracy in the poetry of Walt Whitman	OL18208232W	OL7106853A	Thomas Riggs	5
9	OL27027057M	Contemporary theatre, film and television	OL18208232W	OL7106853A	Thomas Riggs	5
10	OL27564969M	Discover the Savage World	OL9592680W	OL1394228A	Camilla De la Bédoyère	4
11	OL35354818M	Beasties in my backyard	OL9592680W	OL1394228A	Camilla De la Bédoyère	4
12	OL35362368M	Baudolino	OL15653145W	OL20735A	Umberto Eco	3

Total rows: 259 Query complete 00:00:00.182 CRLF Ln 124, Col 1

File Object Tools Edit View Window Help

Welcome perfect last.sql* x hhhhhh.sql* x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr...

Second DB/postgres@Projects

Query Query History

```
select *from fact_book_ratings;
```

Data Output Messages Notifications

Showing rows: 1 to 259 Page No: 1 of 1

	book_id character varying	title character varying (255)	work_id character varying	author_id character varying	author_name character varying (255)	rating numeric
1	OL11353750M	Sacramentary	OL7893135W	OL2589641A	Catholic	3
2	OL11466390M	What the Bible is All about	OL17048540W	OL20625A	Henrietta C. Mears	1
3	OL15233460M	De superman au surhomme	OL15653145W	OL20735A	Umberto Eco	3
4	OL15233460M	De superman au surhomme	OL15653145W	OL20735A	Umberto Eco	4
5	OL17286871M	Yanlis okumalar	OL15653145W	OL20735A	Umberto Eco	3
6	OL17286871M	Yanlis okumalar	OL15653145W	OL20735A	Umberto Eco	4
7	OL25065956M	Strongest and weakest	OL9592680W	OL1394228A	Camilla De la Bédoyère	4
8	OL25299695M	Democracy in the poetry of Walt Whitman	OL18208232W	OL7106853A	Thomas Riggs	5
9	OL27027057M	Contemporary theatre, film and television	OL18208232W	OL7106853A	Thomas Riggs	5
10	OL27564969M	Discover the Savage World	OL9592680W	OL1394228A	Camilla De la Bédoyère	4
11	OL35354818M	Beasties in my backyard	OL9592680W	OL1394228A	Camilla De la Bédoyère	4
12	OL35362368M	Baudolino	OL15653145W	OL20735A	Umberto Eco	3

Total rows: 259 Query complete 00:00:00.182 CRLF Ln 124, Col 1

File Object Tools Edit View Window Help

Welcome perfect last.sql* x hhhhhh.sql* x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr...

Second DB/postgres@Projects

Query Query History Scratch Pad x

```
1 select * from dim_works ;
2 select * from dim_authors;
3 select *from dim_ratings;
4 select * from dim_editions;
5 select *from fact_book_ratings;
6
7
8 --relating authors works
9 SELECT *
10 FROM dim_authors
11 JOIN dim_works
12 ON dim_authors.author_id = dim_works.authors;
13
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 377

	type	work_id	revision	last_modified	title
	text	[PK] text	bigint	timestamp without time zone	text
1	work	OL10000115W	4	2019-03-10 02:42:04.184678	Litterature et reportage
2	work	OL10000144W	2	2020-12-05 02:23:24.753437	Safar fi wujuh
3	work	OL10000169W	3	2010-04-28 06:54:19.472104	Mathématiques, analyse, MP (cours, méthodes, exercices résolus)
4	work	OL10000206W	5	2011-11-25 06:14:28.313274	Grammaire critique de l'italien, volume 13
5	work	OL10000376W	3	2010-04-28 06:54:19.472104	Comme des garçons
6	work	OL10000435W	5	2021-02-24 05:15:17.154746	Ces grands discours qui ont fait le discours
7	work	OL10000494W	3	2010-04-28 06:54:19.472104	Lorie de A à Z
8	work	OL10000606W	7	2022-07-26 07:11:15.619585	L'Avant-scène Opéra, numéros 135-136
9	work	OL10000664W	3	2010-04-28 06:54:19.472104	La nutrition. médecine des suppléments alimentaires
10	work	OL10000706W	4	2022-09-14 15:47:04.700457	C'est la robe

Total rows: 376715 Query complete 00:00:02.120 CRLF Ln 1, Col 1

File Object Tools Edit View Window Help

Welcome perfect last.sql* x hhhhhh.sql* x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr...

Second DB/postgres@Projects

Query Query History Scratch Pad x

```
1 select * from dim_works ;
2 select * from dim_authors;
3 select *from dim_ratings;
4 select * from dim_editions;
5 select *from fact_book_ratings;
6
7
8 --relating authors works
9 SELECT *
10 FROM dim_authors
11 JOIN dim_works
12 ON dim_authors.author_id = dim_works.authors;
13
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 432

	type	book_id	revision	last_modified	title
	text	[PK] text	bigint	timestamp without time zone	text
1	edition	OL10000241M	3	2010-04-13 02:52:51.925831	The National Register of Apparel Manufacturers, Men & Boys
2	edition	OL10000488M	3	2011-04-28 06:17:32.348931	Public Processions Etc. (Northern Ireland) Bill [H.L.]
3	edition	OL10000723M	4	2022-07-17 04:38:18.718745	Crime and Disorder Bill [H.L.]
4	edition	OL10001042M	3	2011-04-28 19:27:40.205073	Northern Ireland Bill
5	edition	OL1000267M	11	2024-08-06 14:00:42.514369	The psalm killer
6	edition	OL10002693M	3	2011-06-08 04:28:03.025393	Health Authorities Bill
7	edition	OL10003167M	3	2011-04-28 00:52:09.260425	Nursery Education and Grant-Maintained Schools Bill
8	edition	OL10003490M	7	2022-07-17 05:19:12.575779	Arbitration Bill [Lords] (Parliamentary Debates: [1995-96])
9	edition	OL10003761M	4	2022-07-17 07:01:12.718767	Finance Bill (Except Clauses 2, 28 and 99) (Parliamentary Debates: [1998-99])
10	edition	OL10004076M	4	2022-07-17 07:01:03.810777	Welfare Reform and Domicile Bill (Parliamentary Debates: [1998-99])

Total rows: 431083 Query complete 00:00:02.693 CRLF Ln 4, Col 28

pgAdmin 4

Welcome perfect last.sql* x hhhhhh.sql* x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr...

Second DB/postgres@Projects

Query Query History

```
1 select * from dim_works ;
2 select * from dim_authors;
3 select *from dim_ratings;
4 select * from dim_editions;
5 select *from fact_book_ratings;
6
7
8 --relating authors works
9 SELECT *
10 FROM dim_authors
11 JOIN dim_works
12 ON dim_authors.author_id = dim_works.authors;
13
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 500

	type	author_id	revision	created_at	name	key	source_records
	text	[PK] text	bigint	timestamp without time zone	text	text	text
1	/type/author	OL10000226A	1	2021-12-26 21:42:09.238563	Alan R. Blackmer	OL10000226A	(bwb:9780674593473)
2	/type/author	OL10000440A	1	2021-12-26 22:24:12.304187	Carmen Mellizo Sanz	OL10000440A	(bwb:9788490150221)
3	/type/author	OL10000917A	1	2021-12-26 23:32:38.430686	Tom Popkewitz	OL10000917A	(bwb:9781607527602)
4	/type/author	OL10001265A	1	2021-12-27 00:25:38.211432	Fan Chengda	OL10001265A	(bwb:9780295802060)
5	/type/author	OL10001270A	1	2021-12-27 00:25:45.312956	Mario R. Dinunzio	OL10001270A	(bwb:9781604264043)
6	/type/author	OL10001396A	1	2021-12-27 00:57:39.652969	Thomas S. O'Connor	OL10001396A	(bwb:9781606493557)
7	/type/author	OL10001709A	1	2021-12-27 01:38:04.408499	Jon Vriesacker	OL10001709A	(bwb:9781480371354)
8	/type/author	OL10001836A	1	2021-12-27 01:48:07.23888	Peter Lieuwen	OL10001836A	(bwb:9781476871578)
9	/type/author	OL10002328A	1	2021-12-27 02:48:53.242959	Yvonne Velasquez	OL10002328A	(bwb:9780578158846)
10	/type/author	OL10002401A	1	2021-12-27 03:15:47.849785	Blank	OL10002401A	(bwb:9780578477594761)

Total rows: 499911 Query complete 00:00:01.798

CRLF Ln 2, Col 1

pgAdmin 4

Welcome perfect last.sql* x hhhhhh.sql* x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr... x Second DB/postgr...

Second DB/postgres@Projects

Query Query History

```
1 select * from dim_works ;
2 select * from dim_authors;
3 select *from dim_ratings;
4 select * from dim_editions;
5 select *from fact_book_ratings;
6
7
8 --relating authors works
9 SELECT *
10 FROM dim_authors
11 JOIN dim_works
12 ON dim_authors.author_id = dim_works.authors;
13
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 377

	type	work_id	revision	last_modified	title
	text	[PK] text	bigint	timestamp without time zone	text
1	work	OL10000115W	4	2019-03-10 02:42:04.184678	Litterature et reportage
2	work	OL10000144W	2	2020-12-05 02:23:24.753437	Safar fi wujuh
3	work	OL10000169W	3	2010-04-28 06:54:19.472104	Mathématiques, analyse, MP (cours, méthodes, exercices résolus)
4	work	OL10000206W	5	2011-11-25 06:14:28.313274	Grammaire critique de l'italien, volume 13
5	work	OL10000376W	3	2010-04-28 06:54:19.472104	Comme des garçons
6	work	OL10000435W	5	2021-02-24 05:15:17.154746	Ces grands discours qui ont fait le discours
7	work	OL10000494W	3	2010-04-28 06:54:19.472104	Lorie de A à Z
8	work	OL10000606W	7	2022-07-26 07:11:15.619585	L'avant-scène Opéra, numéros 135-136
9	work	OL10000664W	3	2010-04-28 06:54:19.472104	La nutrithérapie. medecine des suppléments alimentaires
10	work	OL10000764W	4	2022-08-14 15:47:04.790457	C'est la zèle

Total rows: 376715 Query complete 00:00:02.120

CRLF Ln 1, Col 1

2. Basic OLAP and Reporting:

➤ SQL CODES FOR OLAP OPERATIONS

OLAP OPERATIONS

1 ROLL UP

2 DRILL DOWN

3 SLICE

4 DICE

5 PIVOT

➤ SQL CODE FOR OLAP OPERATIONS

1) Roll-Up: Aggregating data from detailed to summary level

- ✓ Reduces the detail level of the data by aggregating it along a dimension.

Example: Summarizing BOOK publication yearly.

```
SELECT EXTRACT(YEAR FROM publish_date) AS year,  
COUNT(*) AS published_books  
FROM dim_editions  
WHERE publish_date IS NOT NULL  
GROUP BY year  
ORDER BY year;
```

The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL code:

```
1. Roll-Up: Aggregating data from detailed to summary level (Year-wise book count)  
Reduces the detail level of the data by aggregating it along a dimension.  
Example: Summarizing sales data yearly.  
  
SELECT EXTRACT(YEAR FROM publish_date) AS year, COUNT(*) AS published_books  
FROM dim_editions  
WHERE publish_date IS NOT NULL  
GROUP BY year  
ORDER BY year;
```

The Data Output pane shows the results of the query:

year	published_books
1748	1
1847	2
1851	1
1852	1
1853	1
1857	1
1860	1
1861	1
1863	1
1864	1
1866	1

The status bar at the bottom indicates "Total rows: 108" and "Query complete 00:00:00.305".

2. Drill-Down: Breaking down data to finer levels

- ✓ Increases the detail level of the data by breaking down aggregated data into finer granularity.

Example: Drill down from year to month for book count

```
SELECT  
EXTRACT(YEAR FROM publish_date) AS year,  
EXTRACT(MONTH FROM publish_date) AS month,  
COUNT(*) AS published_books
```



```

FROM dim_editions
WHERE publish_date IS NOT NULL
GROUP BY year, month
ORDER BY year, month;

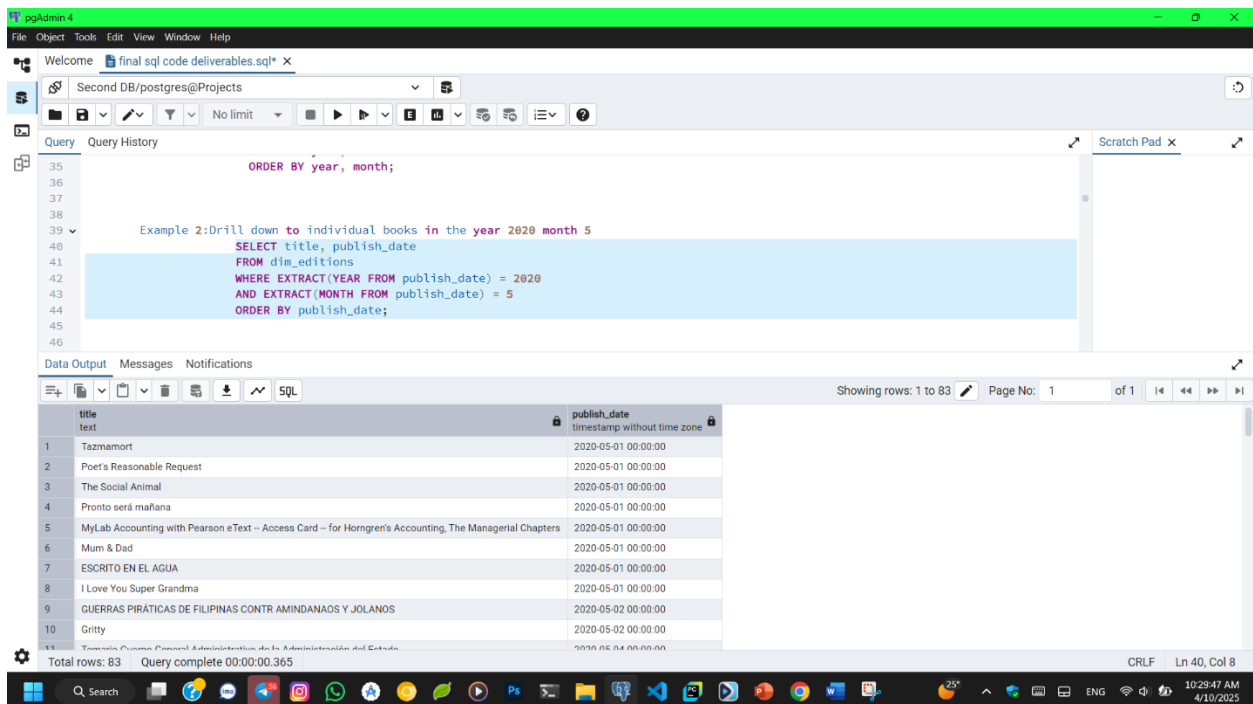
```

Example 2: Drill down to individual books in the year 2020 month 5

```

SELECT title, publish_date
FROM dim_editions
WHERE EXTRACT(YEAR FROM publish_date) = 2020
AND EXTRACT(MONTH FROM publish_date) = 5
ORDER BY publish_date;

```



3. Slice: Filtering data on one dimension

Ex1 :Slice by specific year (e.g., 2018)

```

SELECT book_id,title,authors,publish_date,publishers
FROM publish_date) = 2018
ORDER BY book_id;

```

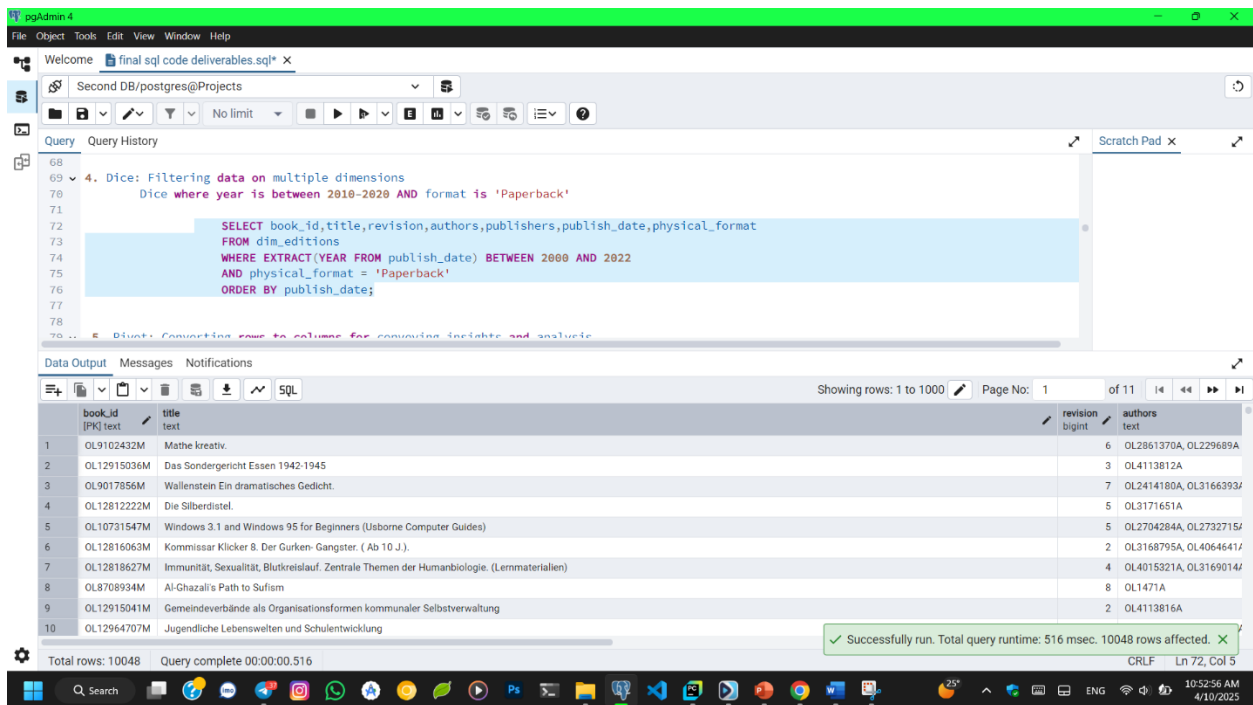
Ex 2: Slice by physical format = 'Hardcover'

```
SELECT book_id,title,authors,publishers,physical_format
FROM dim_editions
WHERE physical_format = 'Hardcover'
ORDER BY book_id;
```

4. Dice: Filtering data on multiple dimensions

EX Dice where year is between 2010–2020 AND format is 'Paperback'

```
SELECT book_id,title,revision,authors,publishers,publish_date,physical_format
FROM dim_editions
WHERE EXTRACT(YEAR FROM publish_date) BETWEEN 2000 AND 2022
AND physical_format = 'Paperback'
ORDER BY publish_date;
```



The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL query:

```
4. Dice: Filtering data on multiple dimensions
Dice where year is between 2010-2020 AND format is 'Paperback'

SELECT book_id,title,revision,authors,publishers,publish_date,physical_format
FROM dim_editions
WHERE EXTRACT(YEAR FROM publish_date) BETWEEN 2000 AND 2022
AND physical_format = 'Paperback'
ORDER BY publish_date;
```

The Data Output pane shows the results of the query, displaying 10 rows of data. The columns are: book_id [PK] text, title text, revision bigint, and authors text. The status bar at the bottom indicates: Total rows: 10048, Query complete 00:00:00.516, and a successful run message: Successfully run. Total query runtime: 516 msec. 10048 rows affected.

book_id [PK] text	title text	revision bigint	authors text
OL9102432M	Mathe kreativ.	6	OL2861370A, OL229689A
OL12915036M	Das Sondergericht Essen 1942-1945	3	OL4113812A
OL9017856M	Wallenstein Ein dramatisches Gedicht.	7	OL2414180A, OL3166393A
OL12812222M	Die Silberdistel.	5	OL3171651A
OL10731547M	Windows 3.1 and Windows 95 for Beginners (Usborne Computer Guides)	5	OL2704284A, OL2732715A
OL12816063M	Kommissar Klicker 8. Der Gurken- Gangster. (Ab 10 J.).	2	OL3168795A, OL4064641A
OL12818627M	Immunität, Sexualität, Blutkreislauf. Zentrale Themen der Humanbiologie. (Lernmaterialien)	4	OL4015321A, OL3169014A
OL8708934M	Al-Ghazali's Path to Sufism	8	OL11471A
OL12915041M	Gemeindeverbände als Organisationsformen kommunaler Selbstverwaltung	2	OL4113816A
OL12964707M	Jugendliche Lebenswelten und Schulentwicklung		

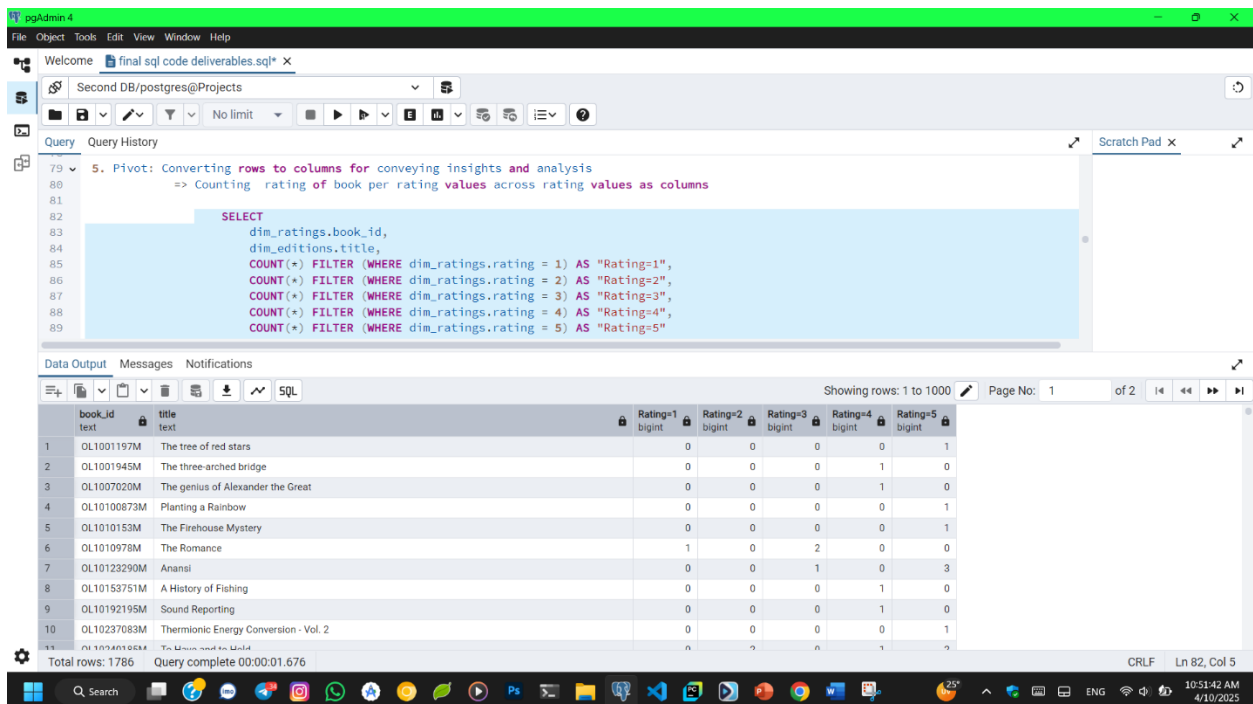
5. Pivot: Converting rows to columns for conveying insights and analysis

=> Counting rating of book per rating values across rating values as columns

```

SELECT
    dim_ratings.book_id,
    dim_editions.title,
    COUNT(*) FILTER (WHERE dim_ratings.rating = 1) AS "Rating=1",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 2) AS "Rating=2",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 3) AS "Rating=3",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 4) AS "Rating=4",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 5) AS "Rating=5"
FROM dim_ratings
JOIN dim_editions ON dim_ratings.book_id = dim_editions.book_id
GROUP BY dim_ratings.book_id, dim_editions.title
ORDER BY dim_ratings.book_id;

```



pgAdmin 4

Second DB/postgres@Projects

Query: 5. Pivot: Converting rows to columns for conveying insights and analysis
=> Counting rating of book per rating values across rating values as columns

```

SELECT
    dim_ratings.book_id,
    dim_editions.title,
    COUNT(*) FILTER (WHERE dim_ratings.rating = 1) AS "Rating=1",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 2) AS "Rating=2",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 3) AS "Rating=3",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 4) AS "Rating=4",
    COUNT(*) FILTER (WHERE dim_ratings.rating = 5) AS "Rating=5"

```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 2

book_id	title	Rating=1	Rating=2	Rating=3	Rating=4	Rating=5
OL1001197M	The tree of red stars	0	0	0	0	1
OL1001945M	The three-arched bridge	0	0	0	1	0
OL1007020M	The genius of Alexander the Great	0	0	0	1	0
OL10100873M	Planting a Rainbow	0	0	0	0	1
OL1010153M	The Firehouse Mystery	0	0	0	0	1
OL1010978M	The Romance	1	0	2	0	0
OL10123290M	Anansi	0	0	1	0	3
OL10153751M	A History of Fishing	0	0	0	1	0
OL10192195M	Sound Reporting	0	0	0	1	0
OL10237083M	Thermionic Energy Conversion - Vol. 2	0	0	0	0	1

Total rows: 1786 Query complete 00:00:01.676

➤ SQL codes for Aggregation (SUM, AVG, COUNT)

- 1 COUNT
- 2 SUM
- 3 AVG

1 COUNT: To count how many books were published in each year, based on the publish_date:

```

SELECT
    EXTRACT(YEAR FROM publish_date) AS publish_year, COUNT(*) AS books_per_year

```

```

FROM dim_editions

GROUP BY publish_year

ORDER BY publish_year;

```

2) SUM: To calculate the sum of ratings for each book:

```

SELECT dim_editions.book_id, dim_editions.title,

SUM(dim_ratings.rating) AS total_rating

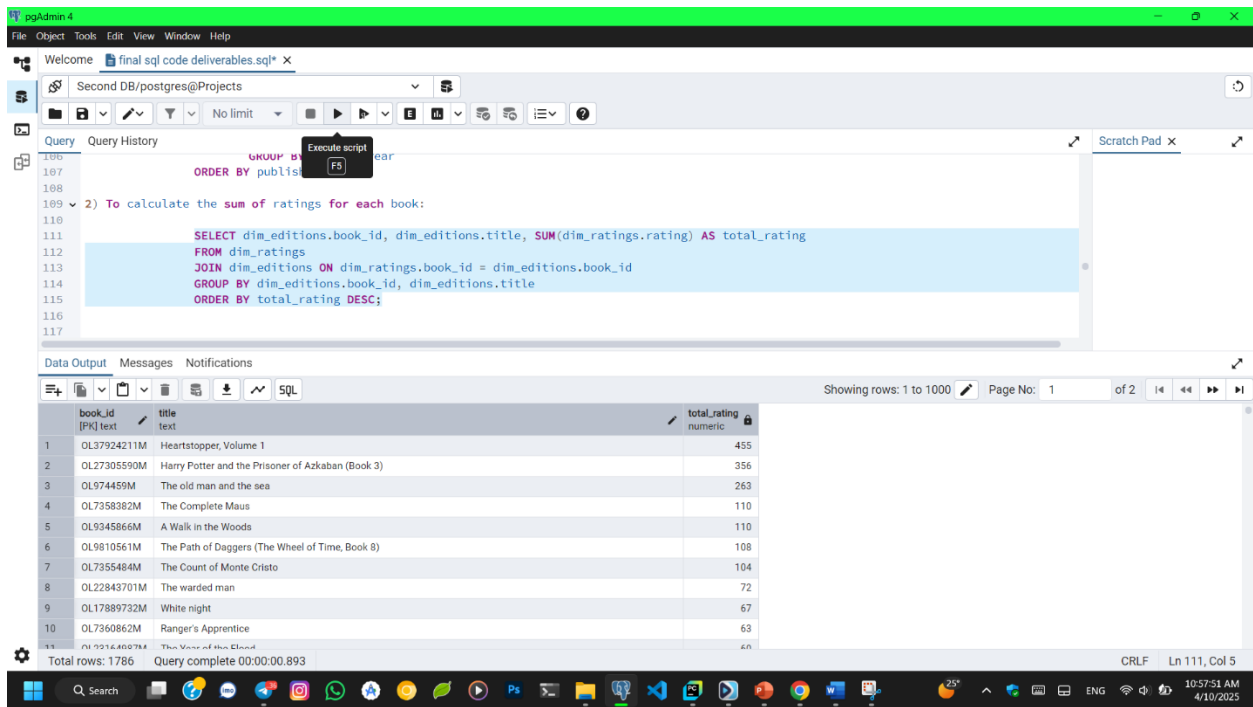
FROM dim_ratings

JOIN dim_editions ON dim_ratings.book_id = dim_editions.book_id

GROUP BY dim_editions.book_id, dim_editions.title

ORDER BY total_rating DESC;

```



The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL query:

```

-- 2) To calculate the sum of ratings for each book:
SELECT dim_editions.book_id, dim_editions.title, SUM(dim_ratings.rating) AS total_rating
FROM dim_ratings
JOIN dim_editions ON dim_ratings.book_id = dim_editions.book_id
GROUP BY dim_editions.book_id, dim_editions.title
ORDER BY total_rating DESC;

```

The Data Output pane shows the results of the query, sorted by total_rating in descending order:

book_id [PK] text	title text	total_rating numeric
OL37924211M	Heartstopper, Volume 1	455
OL27305590M	Harry Potter and the Prisoner of Azkaban (Book 3)	356
OL974459M	The old man and the sea	263
OL7358382M	The Complete Maus	110
OL9345866M	A Walk in the Woods	110
OL9810561M	The Path of Daggers (The Wheel of Time, Book 8)	108
OL7355484M	The Count of Monte Cristo	104
OL22843701M	The warded man	72
OL17889732M	White night	67
OL7360862M	Ranger's Apprentice	63

The interface also shows the status bar at the bottom: Total rows: 1786, Query complete 00:00:00.893, CRLF, Ln 111, Col 5.

3) Average Number of Books Published per Year from 2010 to 2022

```

SELECT

Publication_year,

Total_books,

AVG(total_books) OVER () AS Average_published_book

FROM

```

SELECT

EXTRACT(YEAR FROM publish_date) AS Publication_year,

COUNT(DISTINCT book_id) AS Total_books

FROM dim_editions

WHERE EXTRACT(YEAR FROM publish_date) BETWEEN 2010
AND 2022

GROUP BY EXTRACT(YEAR FROM publish_date)

ORDER BY publication_year

)

ORDER BY publication_year;

The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL query:

```
SELECT
Publication_year,
Total_books,
AVG(total_books) OVER () AS Average_published_book
FROM (
SELECT
EXTRACT(YEAR FROM publish_date) AS Publication_year,
COUNT(DISTINCT book_id) AS Total_books
FROM dim_editions
WHERE EXTRACT(YEAR FROM publish_date) BETWEEN 2010 AND 2022
```

The Data Output pane shows the results of the query:

publication_year numeric	total_books bigint	average_published_book numeric
2010	104	140.9230769230769231
2011	85	140.9230769230769231
2012	112	140.9230769230769231
2013	105	140.9230769230769231
2014	105	140.9230769230769231
2015	122	140.9230769230769231
2016	173	140.9230769230769231
2017	231	140.9230769230769231
2018	232	140.9230769230769231
2019	275	140.9230769230769231

The status bar at the bottom indicates: "Total rows: 13 Query complete 00:00:00.346". A green message box says: "Successfully run. Total query runtime: 346 msec. 13 rows affected."



Filtering Using WHERE

1) selecting books published after 2015

SELECT

book_id,

title,

publish_date

```
FROM dim_editions

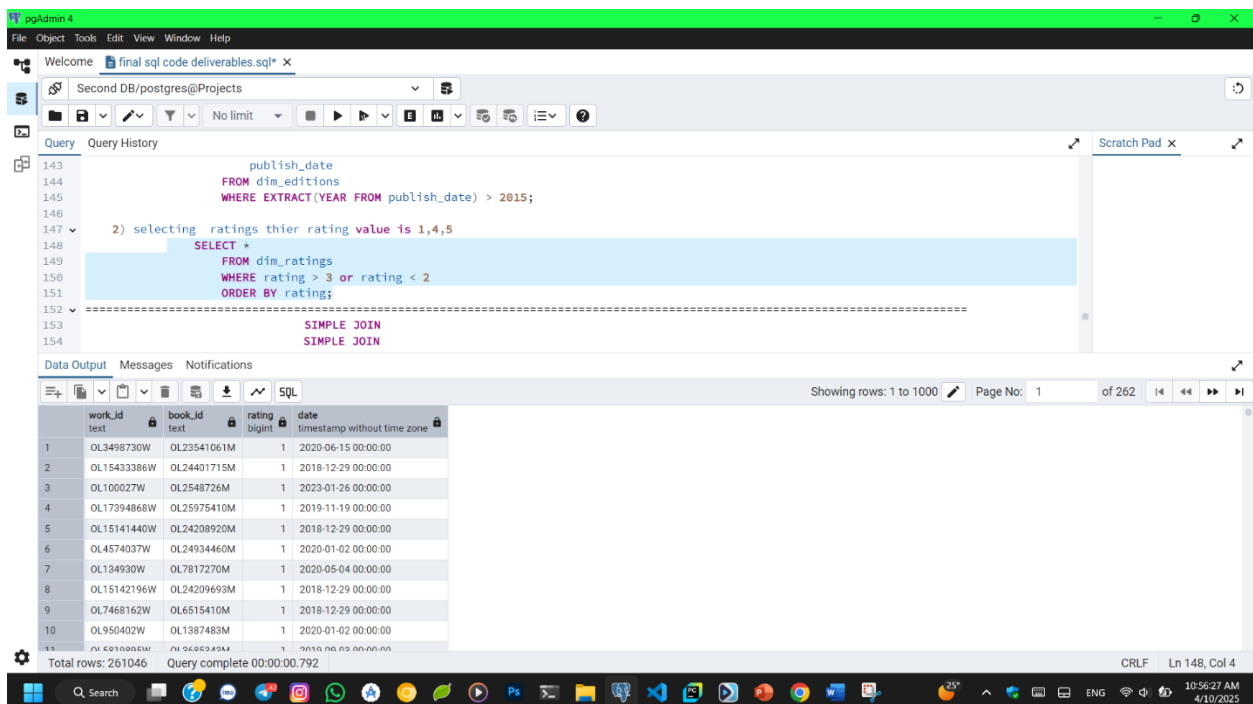
WHERE EXTRACT(YEAR FROM publish_date) > 2015;
```

2) selecting ratings thier rating value is 1,4,5

```
SELECT *FROM dim_ratings

WHERE rating > 3 or rating < 2

ORDER BY rating;
```



The screenshot shows the pgAdmin 4 interface with two SQL queries entered in the 'Query' tab. The first query filters books published after 2015 from the 'dim_editions' table. The second query selects all ratings from the 'dim_ratings' table where the rating is greater than 3 or less than 2, ordered by rating. The 'Data Output' tab shows the results of the second query, displaying a table with columns: work_id, book_id, rating, and date. The results show 10 rows of data.

work_id	book_id	rating	date
1	OL3498730W	OL23541061M	1 2020-06-15 00:00:00
2	OL15433386W	OL24401715M	1 2018-12-29 00:00:00
3	OL100027W	OL2548726M	1 2023-01-26 00:00:00
4	OL17394868W	OL25975410M	1 2019-11-19 00:00:00
5	OL15141440W	OL24208920M	1 2018-12-29 00:00:00
6	OL4574037W	OL24934460M	1 2020-01-02 00:00:00
7	OL134930W	OL7817270M	1 2020-05-04 00:00:00
8	OL15142196W	OL24209693M	1 2018-12-29 00:00:00
9	OL7468162W	OL6515410M	1 2018-12-29 00:00:00
10	OL950402W	OL1387483M	1 2020-01-02 00:00:00

Simple Joins.

1: JOIN uses to Filtering Books Published After 2000 with Ratings

```
SELECT

de.book_id,
```

```

de.title,

dr.rating

FROM dim_editions de

JOIN dim_ratings dr ON de.book_id = dr.book_id

WHERE EXTRACT(YEAR FROM de.publish_date) > 2000;

```

2) : JOIN to detect book with its author id and name the rating value

```

SELECT DISTINCT dim_ratings.book_id,dim_editions.title, dim_authors.author_id,
dim_authors.name,dim_ratings.rating

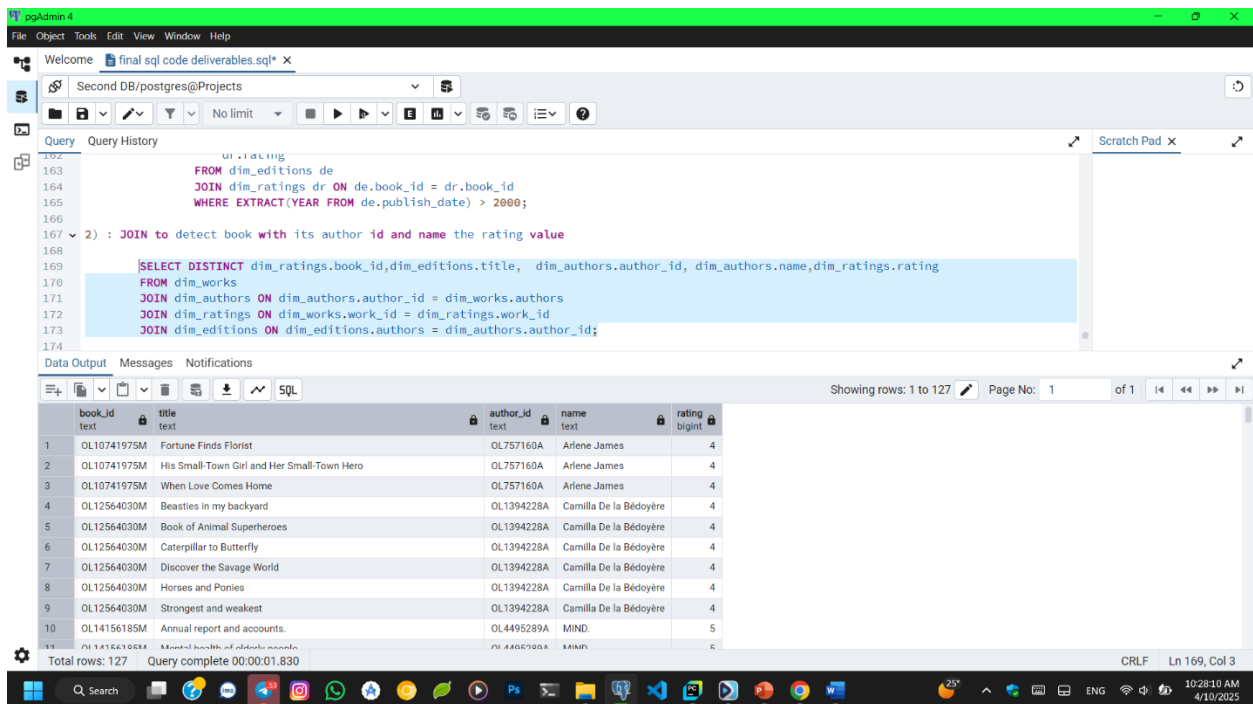
FROM dim_works

JOIN dim_authors ON dim_authors.author_id = dim_works.authors

JOIN dim_ratings ON dim_works.work_id = dim_ratings.work_id

JOIN dim_editions ON dim_editions.authors = dim_authors.author_id;

```



The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL code:

```

-- dr.rating
FROM dim_editions de
JOIN dim_ratings dr ON de.book_id = dr.book_id
WHERE EXTRACT(YEAR FROM de.publish_date) > 2000;

2) : JOIN to detect book with its author id and name the rating value

SELECT DISTINCT dim_ratings.book_id,dim_editions.title, dim_authors.author_id, dim_authors.name,dim_ratings.rating
FROM dim_works
JOIN dim_authors ON dim_authors.author_id = dim_works.authors
JOIN dim_ratings ON dim_works.work_id = dim_ratings.work_id
JOIN dim_editions ON dim_editions.authors = dim_authors.author_id;

```

The Data Output tab shows the results of the query, displaying 127 rows. The columns are: book_id, title, author_id, name, and rating. The results are as follows:

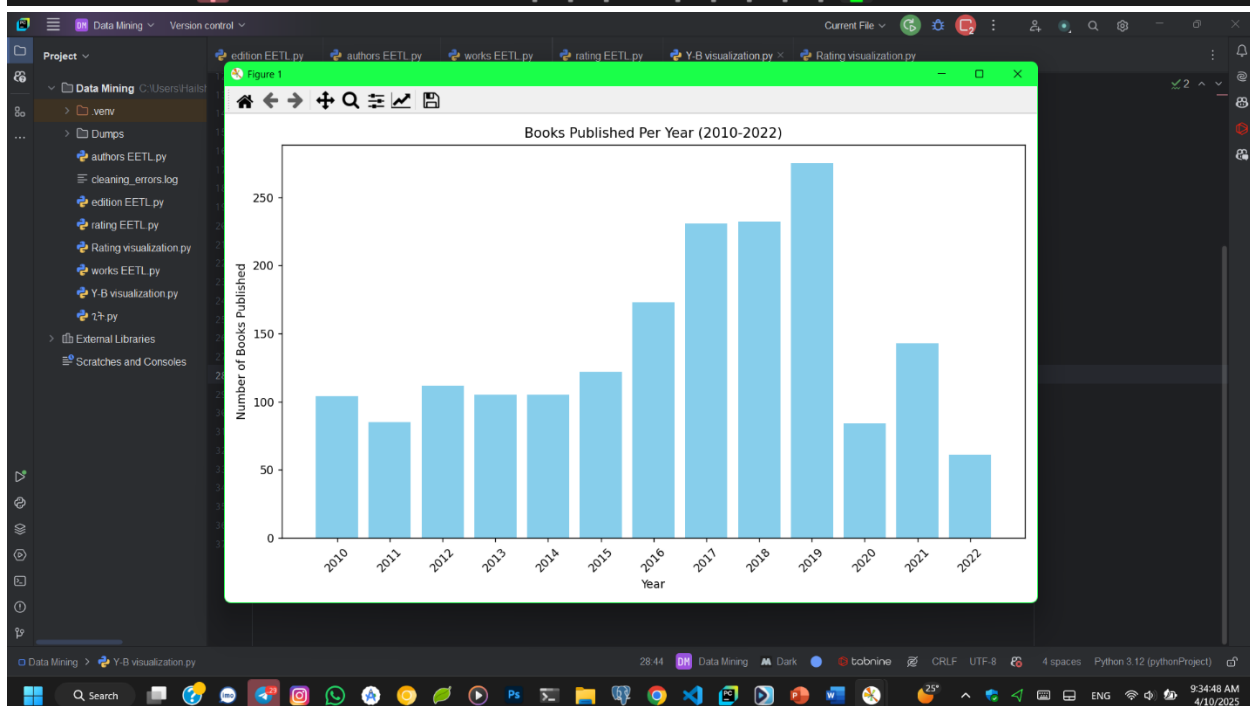
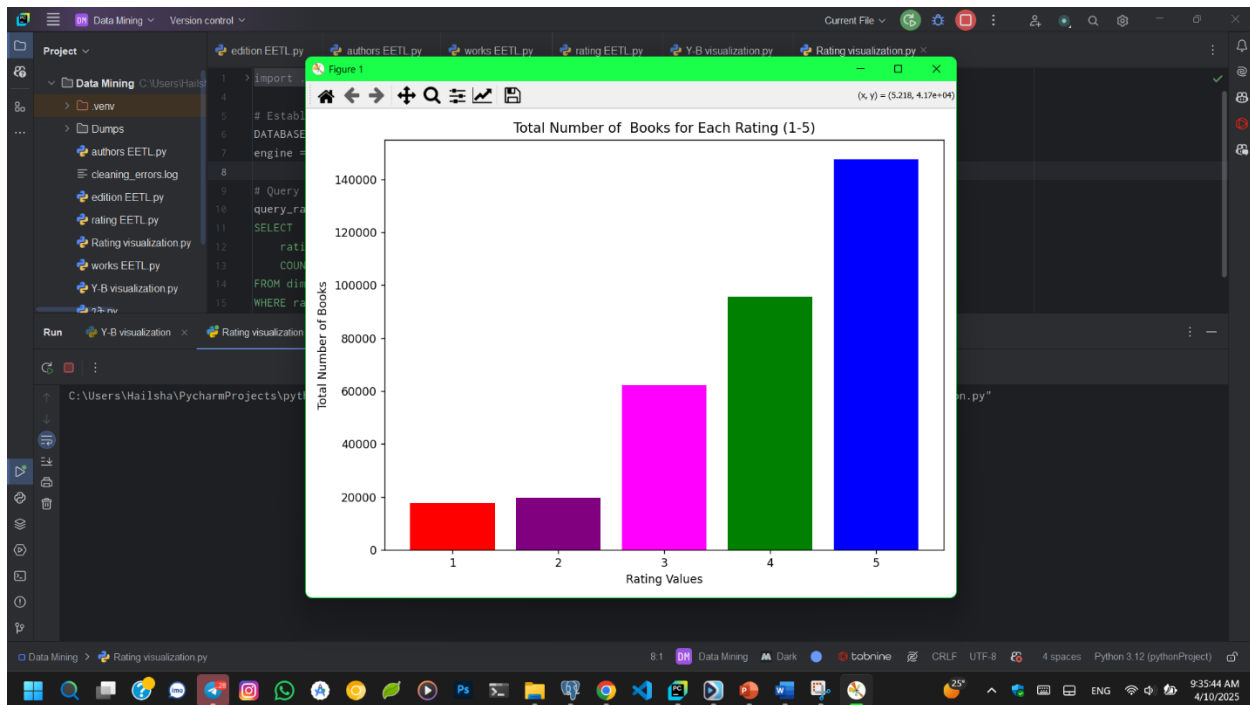
book_id	title	author_id	name	rating
OL10741975M	Fortune Finds Florist	OL757160A	Arlene James	4
OL10741975M	His Small-Town Girl and Her Small-Town Hero	OL757160A	Arlene James	4
OL10741975M	When Love Comes Home	OL757160A	Arlene James	4
OL12564030M	Beasties in my backyard	OL1394228A	Camilla De la Bédoyère	4
OL12564030M	Book of Animal Superheroes	OL1394228A	Camilla De la Bédoyère	4
OL12564030M	Caterpillar to Butterfly	OL1394228A	Camilla De la Bédoyère	4
OL12564030M	Discover the Savage World	OL1394228A	Camilla De la Bédoyère	4
OL12564030M	Horses and Ponies	OL1394228A	Camilla De la Bédoyère	4
OL12564030M	Strongest and weakest	OL1394228A	Camilla De la Bédoyère	4
OL14156185M	Annual report and accounts.	OL4495289A	MIND.	5

Total rows: 127. Query complete 00:00:01.830.

➤ Simple visualizations

✓ using Python/Matplotlib

- Books published from 2010 to 2022
- Total books per ratings(1-5)
- Run the visualization code for visualization



Insights from the visualizations



Publication Trend: Sharp rise in books between 2016–2019.



Rating Insights: Most books received 3–5 stars.